ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

Государственное образовательное учреждение высшего
профессионального образования

«Белгородский государственный технологический университет
им. В.Г. Шухова»

Институт ИТУС
Кафедра информационных технологий

КУРСОВАЯ РАБОТА
по дисциплине «Инфокоммуникационные системы и сети»
на тему:

Разработка сетевой компьютерной игры «Русские шашки»

Выполнил: студент группы ИТЗ-221
Дмитриев Дмитрий Анатольевич
Проверил: ассистент кафедры ИТ
Курбатова Софья Андреевна

Белгород 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1. ПОСТАНОВКА ЗАДАЧИ И ТРЕБОВАНИЯ 4
1.1. Основание для разработки 4
1.2. Назначение программы 5
1.3. Функциональные требования, условия эксплуатации и программная совместимость 6
2. ПРОЕКТИРОВАНИЕ ПРОГРАММЫ 8
2.1. Структура программы и взаимодействие классов 8
2.2. Представление игрового состояния 10
2.3. Алгоритмы ходов, взятий, превращения в дамку и определения победителя 12
2.4. Организация TCP-обмена и завершения сессии 16
3. РЕАЛИЗАЦИЯ ПРОГРАММЫ 21
3.1. Выбор C#, Windows Forms, .NET и Rider 21
3.2. Интерфейс и последовательность действий игроков 22
ЗАКЛЮЧЕНИЕ 26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 27

ВВЕДЕНИЕ

Игра между двумя людьми предполагает общее представление о положении фигур и очередности действий. При игре за одной доской это представление формируется непосредственно: оба участника видят каждое перемещение. Если игроки находятся за разными компьютерами, их действия требуется передавать по сети и воспроизводить на другом устройстве. При этом недостаточно показать изображение доски. Программа должна связывать действия пользователя с правилами игры, поддерживать очередность и согласованно завершать партию.

В курсовой работе рассмотрел задачу создания сетевой компьютерной игры «Русские шашки». Предполагаемый сценарий применения состоит в проведении партии между двумя пользователями Windows, компьютеры которых имеют сетевую доступность. Один участник создаёт игру и ожидает соперника, другой указывает адрес и порт и подключается. Оба используют одно и то же приложение, поэтому отдельная программа для сервера не требуется.

Цель работы разработать сетевую компьютерную игру «Русские шашки», обеспечивающую взаимодействие двух игроков, выполнение ходов по заданным правилам и отображение состояния партии на обоих компьютерах.

Для достижения цели определил требования к игровому процессу и соединению, спроектировал представление шашек и состояния хода, реализовал интерфейс, обычные перемещения и взятия. Отдельно выделил превращение в дамку и продолжение взятия после превращения. Для связи двух экземпляров программы организовал TCP-обмен, а для окончания партии предусмотрел определение победителя при отсутствии шашек у соперника, уведомление участников и закрытие соединения.

Приложение разработал на C# с использованием Windows Forms и .NET. В качестве среды разработки использовал JetBrains Rider. Игровое поле оформил как набор кнопок, связанных с объектами шашек. Логику перемещений разместил в классе Game, операции соединения и передачи данных в классе Network, а обработчики действий пользователя оставил в классе формы Form1.

1. ПОСТАНОВКА ЗАДАЧИ И ТРЕБОВАНИЯ

1.1. Основание для разработки

Основанием для разработки стала потребность провести партию в шашки между двумя пользователями, которые работают на разных компьютерах. Обычная физическая доска в таком сценарии недоступна обоим участникам одновременно. Если каждый игрок самостоятельно отмечает ходы на своей доске, возникает дополнительная задача передачи координат и контроля соответствия позиций. В компьютерной версии эти операции выполняет приложение, а пользователь выбирает шашку и поле назначения.

Содержательная задача проекта связана с совместным изменением одного игрового состояния. Каждый участник имеет собственное окно программы, однако партия должна оставаться общей. Сделанный на одном компьютере ход должен приводить к такому же расположению шашек на другом. При этом действия двух пользователей нельзя произвольно перемешивать: следующий самостоятельный ход становится доступным только после завершения предыдущего.

В качестве предметной области выбрал русские шашки. Доска содержит 64 клетки, из которых для перемещения используются 32 тёмные. В начале партии каждый участник располагает двенадцатью простыми шашками. Белые начинают игру, после чего участники выполняют ходы по очереди. При описании правил перемещения и взятия использовал официальный материал о шашках-64 [1]. Проект сосредоточен на этих правилах и согласованном условии окончания партии.

Небольшой размер доски позволяет явно представить все её клетки и проследить последствия отдельного хода. Требуется учитывать направление движения простой шашки, возможность взятия назад, обязательность взятия, последовательные прыжки и изменение возможностей фигуры после превращения в дамку. Эти особенности определили состав алгоритмов программы.

Для сетевого варианта также существенно различие между действием пользователя и изменением позиции. Нажатие на собственную шашку только выбирает её и не должно передаваться сопернику как ход. Нажатие на недопустимую клетку не меняет доску. Сетевое сообщение формируется лишь тогда, когда игровая логика приняла перемещение. Такое разграничение позволяет не передавать промежуточные действия интерфейса, не влияющие на партию.

Выбрал сценарий непосредственного соединения двух экземпляров приложения. Пользователь, создающий игру, временно выполняет функции принимающей стороны и одновременно остаётся игроком. Второй участник подключается к нему. После установления соединения оба экземпляра содержат одинаковую игровую логику и обрабатывают последовательность ходов. Постоянно работающий внешний сервис для этого сценария не предусмотрен.

Учебная задача заключается в объединении событийного интерфейса и сетевого взаимодействия. В интерфейсе действия возникают по инициативе пользователя, а входящие данные могут поступить во время ожидания. Разделил обработку этих событий по назначению, сохранив единый способ проверки и применения перемещений. Это позволило связать предметные правила с передаваемыми координатами, не включая графические данные в сетевой обмен.

В границы разработки включил создание и разрыв соединения, начальную расстановку, управление собственными шашками, выполнение взятий, работу дамок и завершение партии. Отдельно определил, что победитель выявляется по отсутствию шашек у соперника. Именно это условие использовал при проектировании и реализации.

1.2. Назначение программы

Программа предназначена для проведения одной партии между двумя пользователями. Каждый участник запускает одинаковый исполняемый файл и выбирает способ входа в игру. Создание партии означает переход в режим ожидания входящего TCP-соединения. Подключение означает обращение к адресу компьютера соперника и указанному порту.

Создающему игру участнику назначаются белые шашки, подключившемуся участнику чёрные. Назначение связано с режимом соединения и не требует отдельного сообщения выбора цвета. Первым ходит участник с белыми шашками. Цвет пользователя и текущий ход отображаются раздельно: первая надпись характеризует участника, вторая показывает, кому разрешено действовать в данный момент.

Основное рабочее пространство программы представляет собой доску 8 x 8. Пользователь выбирает свою шашку, затем свободную клетку назначения. Если перемещение допустимо, программа переносит шашку, учитывает взятие и сообщает координаты другому экземпляру. Соперник видит изменение своей доски и получает возможность выполнить ответный ход, когда предыдущий ход завершён полностью.

Для последовательного взятия один игровой ход может включать несколько отдельных перемещений. В этой ситуации назначение интерфейса состоит не только в показе новой позиции, но и в обозначении продолжения действия. Подсказка сообщает о необходимости продолжить взятие той же шашкой. Участник не получает права выбрать другую фигуру, пока цепочка не завершена.

Программа отображает количество белых и чёрных шашек, оставшихся на доске после завершённого хода. Эти значения характеризуют текущую позицию, а не накопленные очки. Дамка учитывается как одна шашка соответствующего цвета. Превращение изменяет допустимые перемещения объекта, но не увеличивает количество фигур и не создаёт дополнительную единицу в счётчике.

При окончании партии приложение показывает общий результат и его значение для конкретного участника. Сообщение о победе одного цвета одинаково характеризует партию в обоих окнах, а надписи о личной победе и поражении различаются. После завершения сетевой сессии доска остаётся видимой, но дальнейшие перемещения блокируются. Новое соединение начинает новую партию с исходной расстановкой.

Данные партии находятся в памяти работающего приложения. Текущее расположение шашек связано с элементами доски, после создания новой партии оно формируется заново. В проекте не использовал постоянное хранилище или базу данных. Такое назначение соответствует короткой сессии двух участников и не предполагает восстановления позиции после перезапуска программы.

1.3. Функциональные требования, условия эксплуатации и программная совместимость

Требования к игре разделил на обработку позиции, взаимодействие пользователей и управление соединением. Для позиции важна допустимость каждого отдельного действия. Для сетевой части важны установление связи, сохранение порядка сообщений и корректная остановка сессии. Интерфейс должен отражать эти состояния, чтобы пользователь различал ожидание соперника, собственный ход и завершённую партию.

При запуске приложение должно показывать начальную расстановку из 24 шашек, но не принимать игровые перемещения до соединения участников. После успешного подключения состояние партии должно инициализироваться одинаково с обеих сторон. Белые занимают три нижние строки, чёрные три верхние, а две средние строки остаются свободными. Координаты клеток в обоих экземплярах должны иметь одинаковый смысл.

Управление допускается только своими шашками и только в свою очередь. Обычная шашка должна перемещаться на одну свободную клетку вперёд по диагонали. При взятии направление может быть как вперёд, так и назад. Наличие хотя бы одного доступного взятия запрещает обычное перемещение. Если после прыжка остаётся продолжение, игрок должен завершить цепочку той же шашкой.

Дамка должна перемещаться по свободной диагонали на произвольное допустимое расстояние. Взятие дамкой допускается при наличии одной шашки соперника на выбранном участке и свободной клетки за ней. Достижение последней строки простой шашкой должно изменять её состояние на дамку. Если это произошло во время взятия, возможность продолжения оценивается уже с учётом нового состояния.

Сетевой обмен должен передавать координаты начальной и конечной клеток принятого перемещения. Получатель должен полностью прочитать сообщение и только затем применять его к доске. При неполном чтении нельзя интерпретировать оставшиеся в буфере значения как корректный ход. При закрытии соединения управление доской должно прекращаться, а интерфейс возвращаться в состояние, позволяющее создать новую сессию.

Для завершения партии предусмотрено отдельное требование: после удаления последней шашки одного цвета оба участника должны получить результат, а сетевые ресурсы должны освобождаться. Сообщение об окончании не заменяет само последнее перемещение. Сначала другая сторона применяет ход, затем подтверждает завершение. Такой порядок позволяет отделить передачу последней позиции от закрытия соединения.

Для интерфейса выбрал фиксированное расположение элементов. Доска занимает квадратную область, а сведения о подключении и партии размещены справа. Это упрощает связь между экранным положением кнопки и координатами клетки. Рабочая область экрана должна позволять отображать окно целиком без потери нижних элементов управления.

Текущий проект ориентирован на Windows, поскольку использует Windows Forms. Эта библиотека предоставляет оконные элементы управления и событийную модель настольного приложения [2]. В проекте задана целевая платформа net10.0-windows. Для запуска версии, зависящей от установленной среды, на компьютере

нужна совместимая среда .NET Desktop Runtime, конкретный состав распространяемых файлов зависит от способа публикации проекта.

Для соединения требуется доступность принимающего компьютера по выбранному TCP-порту. По умолчанию в интерфейсе указан порт 5000. Пользователь может изменить его, но значения на обеих сторонах должны совпадать. Принимающая программа должна получить разрешение на входящее соединение в используемой сети. Наличие сетевого адреса само по себе не гарантирует доступности порта при ограничениях брандмауэра.

Адрес 127.0.0.1 в поле подключения соответствует тому же компьютеру и подходит для работы двух экземпляров на одном устройстве. Для партии на двух компьютерах подключающийся участник указывает адрес компьютера, создавшего игру. В рассматриваемой конфигурации принимающая сторона использует IPv4. Автоматического поиска соперника и автоматической настройки сетевого оборудования в проекте нет.

Программа проверяет корректность числового порта и формата IP-адреса до попытки подключения. Время ожидания подключения клиента ограничено десятью секундами. Неудача соединения сопровождается сообщением о необходимости проверить адрес, порт и доступность второго игрока. Это позволяет отличать сетевую ошибку от запрета игрового хода, который возникает уже внутри установленной сессии.

2. ПРОЕКТИРОВАНИЕ ПРОГРАММЫ

2.1. Структура программы и взаимодействие классов

Программу организовал как одно приложение Windows Forms. Точка входа создаёт основную форму, которая содержит игровое поле, параметры соединения и сведения о партии. Разделил реализацию на четыре основных класса: Form1, Game, Network и Checker. Такое деление отражает назначение кода: обработку событий окна, правила игры, сетевую сессию и свойства отдельной шашки.

В классе Form1 находятся реакции на создание игры, подключение, отключение и нажатие клетки. Подключение обработчиков к элементам выполнено в дизайнерской части формы. Сами методы обработчиков размещены в основном файле формы, поэтому настройка внешнего вида и код реакции на событие разделены по файлам одной частичной декларации класса.

В Form1 создал по одному объекту Game и Network. Оба получают ссылку на форму и используют её для обращения к нужным элементам. Это важно для точного описания архитектуры: классы разделены по обязанностям, но остаются связанными с Windows Forms. Game не является полностью независимой от интерфейса моделью, поскольку работает с кнопками доски. Network также обновляет состояния кнопок и текстовые надписи формы.

В Game сосредоточил операции, относящиеся к партии. Он формирует начальную расстановку, принимает выбранную клетку, проверяет перемещение, изменяет положение шашек, обрабатывает взятия и определяет результат. Класс также обновляет сведения о ходе и количестве фигур. Благодаря этому обработчик нажатия в форме не содержит отдельной реализации правил для каждого вида шашки.

В Network осуществляется создание принимающего сокета, подключение к сопернику, чтение и отправку сообщений, отмену ожидания и освобождение ресурсов. Он не вычисляет допустимую диагональ и не определяет, какую шашку нужно снять. Полученные координаты передаются форме, а форма обращается к Game. Разделение сохраняет одну реализацию применения хода для локального действия и входящего сообщения.

Checker хранит два свойства: цвет IsWhite и признак дамки IsQueen. Цвет задаётся при создании объекта и затем доступен для чтения. Признак дамки изменяется при достижении соответствующей строки. Сам объект не содержит сетевого соединения, ссылки на игрока или процедуры перемещения. Его назначение состоит в представлении свойств той шашки, которая в данный момент занимает клетку.

Связи основных классов представлены на рисунке 2.1. Форма создаёт Game и Network, а эти классы используют переданную ссылку на форму для взаимодействия с окном.

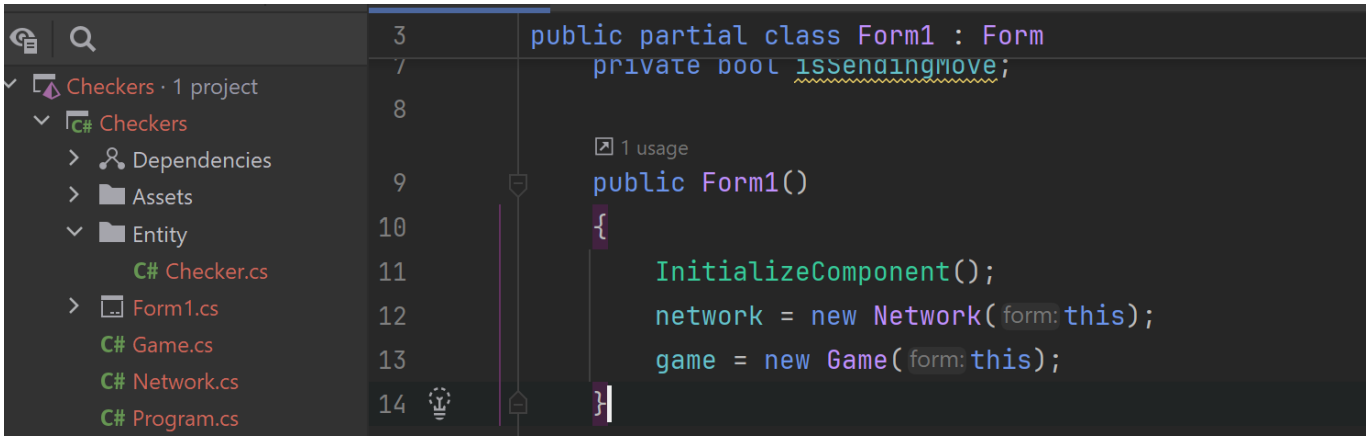


Рисунок 2.1 Структура программы и связи основных классов

Основные обязанности классов сопоставил в таблице 2.1. Такое представление позволяет проверить, где реализована каждая часть поведения, не перечисляя все элементы управления формы.

Таблица 2.1 Назначение основных классов

Класс	Обязанности
Form1	Обработчики событий, вызовы Game и Network, передача входящих ходов и результата
Game	Позиция, правила перемещения и взятия, превращение, очередность, счётчики и победитель
Network	TCP-соединение, сообщения фиксированной длины, отмена операций и завершение сессии
Checker	Цвет шашки и признак её превращения в дамку

Для локального перемещения последовательность начинается с события Click. Форма передаёт выбранную кнопку методу SelectCell. Если нажатие только изменило выбор или не соответствует допустимому действию, метод возвращает отсутствие сообщения. Если произошло допустимое перемещение, Game применяет его и возвращает массив из четырёх байтов. Затем форма передаёт этот массив сетевому классу для отправки.

Промежуток отправки выделил флагом isSendingMove. Пока асинхронная запись не завершилась, обработчик клетки не принимает следующее локальное нажатие как новое действие. При приёме данных направление взаимодействия меняется. Network получает полный массив, вызывает ReceiveMove у формы, а форма передаёт его в Game. Игровой класс проверяет координаты, очередность и допустимость действия, после чего использует ту же процедуру изменения позиции. Полученный от соперника ход не отправляется обратно: сетевой класс уже знает, что обрабатывает входящее сообщение.

Такое разграничение устраняет циклическую пересылку одного перемещения. Если бы любое изменение доски автоматически вызывало отправку, получатель мог бы повторно послать принятый ход.

Начало и остановку партии также провёл через методы формы. Network сообщает форме об успешном соединении и передаёт назначенный цвет. Form1 вызывает StartGame у Game. При отключении форма вызывает StopGame, а при штатном окончании показывает результат. Эти короткие методы не дублируют игровую логику, а определяют точки связи между двумя обслуживающими классами.

Выбранная структура удобна для текущего объёма задачи, поскольку переход от нажатой кнопки к шашке не требует промежуточного слоя отображения. В то же время связь Game с кнопками означает зависимость логики от устройства формы.

2.2. Представление игрового состояния

Игровое состояние представил через кнопки клеток и объекты Checker. Для каждой клетки доски в дизайнере создан отдельный Button. Если клетка занята, её свойство Tag содержит ссылку на объект шашки. Для свободной клетки Tag равен null. Свойство Tag предназначено для хранения связанных с элементом произвольных данных [3], поэтому его использовал как непосредственную связь между графической клеткой и предметным объектом.

Координата относится к кнопке, а объект Checker хранит только цвет и признак дамки. Для определения позиции используется положение элемента в панели доски. Строка вычисляется делением вертикальной координаты кнопки на 125, столбец делением горизонтальной координаты на 125. Размер одной клетки составляет 125 на 125 единиц координат интерфейса.

Началом координат является верхняя левая клетка. Строки и столбцы имеют номера от 0 до 7. При такой ориентации белые начинают в нижней части и движутся к уменьшению номера строки, а чёрные в верхней и движутся к увеличению. Одинаковая система координат используется в обоих окнах, поэтому входящее сообщение не требует зеркального преобразования доски.

Для обращения к клетке по полученным координатам использовал единое именование кнопок. Имя содержит номер строки и столбца, что позволяет найти элемент в коллекции панели. Таким образом, связь работает в двух направлениях: локальное нажатие даёт кнопку, из которой вычисляются координаты, а сетевые координаты позволяют получить соответствующую кнопку на другой стороне.

Начальную расстановку создаёт InitializeCheckers. Сначала метод очищает у каждой клетки Tag, фоновое изображение и текст. Затем пропускает светлые клетки и заполняет тёмные в первых и последних трёх строках. Для каждой занятой клетки создаётся новый объект Checker с соответствующим цветом и выключенным признаком дамки. Это исключает перенос свойств фигур из предыдущей партии в новую.

При обычном перемещении в клетку назначения переносится ссылка из Tag исходной клетки. Одновременно переносится изображение, после чего исходная клетка освобождается. Новый объект шашки на каждом ходе не создаётся. Сохраняется тот же объект с прежним цветом и текущим признаком дамки. Такой способ особенно удобен при превращении, поскольку изменённое свойство остаётся связано с перемещаемой фигурой.

Фоновое изображение определяет внешний вид простой шашки, но не является источником её логических свойств. Проверка цвета выполняется по IsWhite, а

проверка возможностей дамки по IsQueen. Это позволяет отличать предметное состояние от оформления. Даже если у белой и чёрной шашек используются разные ресурсы изображения, правила не зависят от сравнения этих ресурсов.

Для дамки поверх изображения вывожу букву «Д». При уходе фигуры исходная клетка очищается не только от изображения и Tag, но и от этой надписи.

Состояние партии не исчерпывается расположением объектов. Дополнительно храню признак начала игры, цвет локального пользователя, цвет текущего хода и возможного победителя. Пока игра не начата, выбор клеток не создаёт перемещений. Цвет пользователя постоянен в пределах сессии, а цвет хода изменяется только после полностью завершённого действия.

Сведения о выбранной шашке и о шашке, продолжающей взятие, разделил. selectedChecker обозначает текущий выбор пользователя. capturingChecker хранит клетку фигуры, которой обязательно продолжать цепочку. До взятия пользователь может менять выбор допустимой собственной шашки. После начала серии свободная смена фигуры запрещена, поэтому состояние цепочки имеет более строгий смысл, чем обычное выделение.

Для уже взятых в текущем ходе фигур использовал HashSet<Button>. В этом множестве хранятся клетки, шашки на которых были перепрыгнуты. До завершения всей серии они остаются занятыми, но не могут быть взяты повторно. После последнего прыжка у всех записанных клеток очищаются Tag, изображение и текст, а множество становится пустым.

Основные данные партии и их назначение приведены в таблице 2.2. Значения существуют в памяти каждого экземпляра программы и не пересылаются полностью после каждого хода.

Таблица 2.2 Представление состояния партии

Данные	Смысл и изменение
Button.Tag	Ссылка на Checker для занятой клетки; null для свободной
IsWhite	Цвет фигуры, задаваемый при создании
IsQueen	Признак дамки, включаемый при достижении последней строки
gameStarted	Разрешение обработки игровых перемещений
isWhitePlayer	Цвет участника данного экземпляра программы
isWhiteTurn	Цвет игрока, выполняющего текущий ход
selectedChecker	Клетка выбранной пользователем шашки
capturingChecker	Клетка шашки, обязанной продолжать взятие
capturedCheckers	Клетки фигур, взятых в незавершённой цепочке
winnerIsWhite	Неопределённый результат либо цвет установленного победителя

Значение победителя использует тип bool с возможностью отсутствия значения. Пока результат не определён, хранится null. Значение true соответствует победе белых, false победе чёрных. Такая форма отделяет отсутствие результата от двух возможных исходов. Обычный логический флаг без третьего состояния не позволил бы различить ещё продолжающуюся партию и уже установленную победу чёрных.

Счётчики на форме вычисляются перебором клеток, содержащих Checker. Отдельный постоянно изменяемый счётчик снятых фигур не храню. Пересчёт после завершения хода позволяет получать значения из фактической позиции.

Фиксированный размер доски здесь является частью вычислительной модели, а не только внешнего оформления. Если изменить размеры или взаимное расположение кнопок без изменения расчёта координат, деление на 125 перестанет корректно определять строку и столбец. Поэтому статическую конфигурацию интерфейса сохранил согласованной с алгоритмом и форматом сообщения.

2.3. Алгоритмы ходов, взятий, превращения в дамку и определения победителя

Проверку перемещения организовал последовательно: сначала определяется право пользователя действовать, затем выбор фигуры, затем допустимость клетки назначения. Это разграничение не позволяет смешивать нажатие кнопки с уже совершённым ходом. Не каждое событие Click изменяет состояние доски, и только принятый переход между клетками становится основанием для сетевой отправки.

В начале SelectCell проверяет, начата ли партия и совпадает ли цвет текущего хода с цветом локального участника. Если хотя бы одно условие не выполнено, перемещение не формируется. Благодаря этому визуально доступная клетка не означает право немедленно использовать её. Основное ограничение очередности находится в игровом классе и действует независимо от выбранной шашки.

При нажатии занятой собственной клетки определяется возможность выбора. Во время цепочки допускается только та шашка, которая уже начала взятие. В обычном состоянии дополнительно проверяется наличие взятий у любой своей фигуры. Если взятие существует, выбрать для хода шашку без доступного взятия нельзя. Это ограничение вводится до попытки переноса и помогает сохранить обязательность действия.

При нажатии пустой клетки метод проверяет наличие выбранной шашки и допустимость перехода. Неудачная попытка не переносит ни объект, ни изображение. Если серия взятий уже началась, выбор возвращается к продолжающей её фигуре. Если серии нет, неудачное действие не создаёт сетевого сообщения. Алгоритм обработки выбора клетки предусмотрел на рисунке 2.1.

```
public byte[]? SelectCell(Button cell)
{
    if (!gameStarted || isWhiteTurn != isWhitePlayer)
    {
        return null;
    }

    if (cell.Tag is Checker checker && checker.IsWhite == isWhitePlayer)
    {
        if (capturingChecker != null && cell != capturingChecker)
        {
            return null;
        }

        if (capturingChecker == null && HasAnyCapture(isWhiteTurn) &&
!HasCapture(cell))
        {
            selectedChecker = null;
            return null;
        }

        selectedChecker = cell;
        return null;
    }

    if (selectedChecker == null || cell.Tag != null ||
!canMoveChecker(selectedChecker, cell))
    {
        selectedChecker = capturingChecker;
        return null;
    }

    byte[] move =
    {
        (byte)(selectedChecker.Top / 125),
        (byte)(selectedChecker.Left / 125),
        (byte)(cell.Top / 125),
        (byte)(cell.Left / 125)
    };

    MoveChecker(selectedChecker, cell);
    return move;
}
```

Рисунок 2.1 Обработка выбора клетки и допустимого перемещения

Для простой шашки обычный ход определяю через разности координат. Разность столбцов по модулю должна равняться единице. Разность строк должна совпадать с направлением цвета: минус единица для белых и плюс единица для чёрных. Клетка назначения предварительно проверяется на отсутствие объекта. Таким образом, перемещение в сторону, назад без взятия или через несколько клеток не проходит эту проверку.

Диагональное движение сохраняет цвет клетки. Если фигура стоит на тёмном поле и строка со столбцом одновременно изменяются на единицу, сумма координат меняется на чётное число. Поэтому фигура остаётся на тёмном поле.

Для дамки обычный ход допускается на большее расстояние. Сначала проверяю, что перемещение не нулевое и модули разностей строки и столбца одинаковы. Затем определяю единичные шаги обхода по знакам этих разностей. Каждая промежуточная клетка проверяется на занятость. Любая фигура на пути запрещает обычное перемещение, поскольку проход через неё требует отдельной проверки взятия.

Разделение обычного хода и взятия необходимо для дамки особенно явно. Само наличие противника на диагонали ещё не делает любое движение по ней допустимым. Нужно удостовериться, что за противником есть свободное поле назначения, а до него и после него нет второй преграды.

Если на участке вообще нет противника, метод не возвращает взятую клетку. Такой переход может быть обычным ходом дамки, но не взятием. Внешняя проверка сначала рассматривает взятие и лишь при его отсутствии решает, допускается ли обычное перемещение. В результате одинаковая геометрическая диагональ может относиться к разным типам действий в зависимости от содержимого промежуточных клеток.

Обязательность взятия реализовал через два уровня поиска. HasCapture перебирает возможные клетки назначения для одной фигуры. HasAnyCapture перебирает собственные шашки текущего цвета и вызывает первый метод. Если обнаружен хотя бы один допустимый вариант, обычный ход запрещается. Перебор завершается при нахождении подходящего действия, поскольку для запрета обычного хода достаточно факта существования взятия.

При нескольких доступных взятиях пользователь выбирает допустимую ветвь. Алгоритм не сравнивает все продолжения по количеству будущих снятых шашек и не вычисляет наиболее выгодную линию. Он проверяет корректность выбранного действия в текущей позиции. Это сохраняет управление решением за игроком и соответствует используемому правилу выбора направления взятия [1].

После принятого перемещения MoveChecker определяет, была ли перепрыгнута шашка, и при необходимости добавляет её клетку в множество взятых. Затем переносит объект и изображение в назначение, освобождает исходную клетку и проверяет превращение. Только после этих действий исследуется возможность следующего взятия. Порядок важен: продолжение должно оцениваться из новой позиции и с актуальными свойствами фигуры.

Взятые в пределах цепочки шашки снимаются после её полного завершения. До этого их клетки остаются занятыми и включены в capturedCheckers. При проверке нового прыжка попадание на уже взятую фигуру приводит к отказу. Такой порядок не позволяет второй раз перепрыгнуть ту же фигуру и не освобождает преждевременно диагональ, которая должна оставаться перекрытой до окончания хода.

Когда продолжение отсутствует, программа перебирает множество взятых клеток и очищает их. После этого очищает само множество и ссылки выбора, переключает цвет текущего хода и обновляет количество шашек. Лишь завершённая серия рассматривается как граница между ходами игроков. Схему этой последовательности предусмотрел на рисунке 2.2.

```
private void MoveChecker(Button from, Button to)
{
    Button? capturedChecker = GetCapturedChecker(from, to);
    if (capturedChecker != null)
    {
        capturedCheckers.Add(capturedChecker);
    }

    Checker checker = (Checker)from.Tag!;
    to.BackgroundImage = from.BackgroundImage;
    to.Tag = from.Tag;
    from.BackgroundImage = null;
    from.Tag = null;
    from.Text = "";

    int row = to.Top / 125;
    if ((checker.IsWhite && row == 0) || (!checker.IsWhite && row == 7))
    {
        checker.IsQueen = true;
    }

    to.Text = checker.IsQueen ? "Д" : "";
    to.ForeColor = checker.IsWhite ? Color.Black : Color.White;

    if (capturedChecker != null && HasCapture(to))
    {
        capturingChecker = to;
        selectedChecker = to;
        UpdateTurn();
        return;
    }

    foreach (Button captured in capturedCheckers)
    {
        captured.Tag = null;
        captured.BackgroundImage = null;
        captured.Text = "";
    }

    capturedCheckers.Clear();
    capturingChecker = null;
    selectedChecker = null;
    isWhiteTurn = !isWhiteTurn;
    UpdateCounts();

    if (!HasCheckers(isWhiteTurn))
    {
        winnerIsWhite = !isWhiteTurn;
        gameStarted = false;
        form.pnlBoard.Enabled = false;
        form.lblTurn.Text = "Партия окончена";
        form.lblHint.Text = "Завершение сессии";
        return;
    }

    UpdateTurn();
}
```

Рисунок 2.2 Выполнение перемещения и завершение цепочки взятий

Превращение выполняется при попадании белой шашки в строку 0 или чёрной в строку 7. Свойство IsQueen устанавливается в true и в кнопке появляется обозначение дамки. Не создаю новую фигуру другого типа, поскольку цвет и положение уже представлены существующим объектом и клеткой. Изменения одного

признака достаточно, чтобы следующие проверки движения использовали правила дамки.

Если последняя строка достигнута обычным ходом, этот ход завершается. Если достижение произошло при взятии, после установки IsQueen вызывается проверка возможного продолжения. Она уже допускает дальнее перемещение дамки по диагонали. Поэтому превращение не откладывается до следующей очереди, когда текущая серия может быть продолжена новым способом движения.

Определение победителя разместил после окончания хода и удаления всех взятых фигур. Сначала очередь переключается на другой цвет. Затем HasCheckers проверяет, осталась ли у этого цвета хотя бы одна шашка. Если ни одного объекта не найдено, победителем становится цвет участника, только что завершившего ход.

После выявления победителя сохраняю его цвет, снимаю признак активной игры и блокирую панель доски. Сначала интерфейс показывает окончание партии и завершение сессии. Итоговое окно выводится после выполнения сетевой процедуры подтверждения и освобождения ресурсов.

Для получения удалённого хода предусмотрел дополнительные проверки входных данных. Массив должен содержать четыре байта, значения координат не должны превышать 7, а текущая очередь должна принадлежать удалённому участнику. Исходная клетка должна содержать шашку нужного цвета, назначение должно быть свободным. После этих проверок применяется общий алгоритм допустимости перемещения.

Повторное использование правил при приёме важно даже при работе двух одинаковых экземпляров. Оно не позволяет непосредственно записать любую полученную координату в интерфейс. Ошибочный ход не изменяет позицию, а сетевая часть завершает соединение с сообщением об ошибке.

Размер доски ограничен 64 клетками, поэтому для поиска взятий использовал простой перебор. Одна проверка рассматривает клетки назначения и промежуточный участок диагонали.

2.4. Организация TCP-обмена и завершения сессии

Для передачи ходов выбрал TCP. Соединение используется в течение одной партии, а данные передаются последовательно через NetworkStream. Классы TcpClient и TcpListener предоставляют доступ к TCP-соединению средствами .NET [4]. В проекте принимающий экземпляр создаёт слушатель порта, а подключающийся экземпляр создаёт клиент. После установления связи обе стороны получают поток для двустороннего обмена.

В OpenConnectionAsync сначала проверяется, что другая операция соединения ещё не выполняется. Затем читается порт из формы и проверяется его диапазон. Для подключающейся стороны дополнительно разбирается IP-адрес. До успешного завершения этих проверок сетевые ресурсы не создаются. При ошибке ввода программа показывает понятное сообщение и оставляет возможность исправить параметры.

Создающая игру сторона использует TcpListener с IPAddress.Any и выбранным портом. Это означает ожидание входящих IPv4-соединений на доступных интерфейсах компьютера. После Start вызывается асинхронное ожидание одного

клиента. Надпись состояния сообщает об ожидании игрока. До его подключения доска не переходит в режим выполнения ходов.

После принятия клиента слушатель останавливается. Установленное соединение при этом представлено отдельным TcpClient и продолжает использоваться для партии. Такой порядок ограничивает сессию двумя участниками и не оставляет режим приёма дополнительных игроков. Остановка слушателя и закрытие принятого соединения являются разными операциями с разным назначением.

Подключающийся экземпляр создаёт TcpClient для семейства разобранного адреса и выполняет ConnectAsync. Для этой операции создаётся связанный источник отмены с ограничением в десять секунд.

Когда соединение установлено, Network назначает цвет участника, меняет состояние на «Подключено» и вызывает начало игры через форму. Параметры подключения на время сессии блокируются. Это не позволяет изменить адрес или порт в интерфейсе так, будто действующее соединение уже использует новые значения.

Для цепочки взятий каждый отдельный прыжок формирует собственное сообщение из четырёх байтов. Граница игрового хода при этом определяется правилами Game, а не числом полученных пакетов. Получатель после первого прыжка может сохранить очередь за тем же цветом и ожидать следующего сообщения от того же участника. Смена очереди происходит только тогда, когда серия полностью закончена.

Особенность TCP состоит в потоковом представлении данных. Один вызов чтения не обязан вернуть все байты прикладного сообщения. Документация NetworkStream.ReadAsync допускает получение меньшего количества данных, чем размер предоставленного буфера [5]. Поэтому нельзя считать, что один вызов записи четырёх байтов всегда соответствует одному чтению четырёх байтов на другой стороне.

В цикле приёма сначала читаю один байт. Нулевое количество прочитанных байтов означает завершение потока до начала следующего сообщения, и цикл заканчивается. Если первый байт получен, оставшиеся три читаются методом ReadExactlyAsync. Этот метод завершает операцию только после заполнения указанного участка либо сообщает о невозможности дочитать его [6]. Игровая обработка начинается после получения всего сообщения.

Такой способ различает закрытие соединения между сообщениями и обрыв внутри сообщения. В первом случае новое действие ещё не началось. Во втором получена только часть координат, поэтому применять её нельзя. Если в поток последовательно поступили несколько сообщений, чтение сохраняет их границы за счёт фиксированной длины. Первый цикл потребляет ровно четыре байта, следующий начинает чтение со следующего байта потока.

Для обычного сообщения Network вызывает форму, а затем Game. Если Game отвергает ход, сетевой класс рассматривает это как ошибку и прекращает сессию. Если действие принято, Network проверяет, появился ли локально вычисленный победитель. При отсутствии результата цикл продолжает ожидание следующего сообщения.

Отправка обычного хода выполняется после локального применения перемещения. Пользователь сразу видит принятую позицию, а затем завершается

асинхронная запись. Если запись не удалась, соединение закрывается и продолжение партии блокируется. Автоматическое восстановление этой позиции на новой сессии не реализовано.

Для подтверждения окончания выделил сообщение той же длины. Первый байт имеет значение 255, которое не может быть номером строки доски. Второй байт кодирует цвет победителя: 1 для белых и 0 для чёрных. Третий и четвёртый байты равны нулю. Структура служебного сообщения приведена в таблице 2.4.

Таблица 2.4 Формат сообщения завершения

Позиция байта	Содержимое	Значение
0	Признак служебного сообщения	255
1	Цвет победителя	1 для белых, 0 для чёрных
2	Неиспользуемая позиция	0
3	Неиспользуемая позиция	0

Текущая обработка подтверждения использует первый байт как признак завершения.иШтатное завершение начинается у участника, сделавшего последнее взятие. Его Game удаляет фигуры, обнаруживает отсутствие шашек другого цвета и блокирует доску. Обычное сообщение с координатами последнего прыжка всё равно отправляется. Второй экземпляр получает его, применяет и также вычисляет победителя. После этого он посылает четырёхбайтовое подтверждение и выходит из цикла чтения.

Первый экземпляр получает признак 255, извлекает уже вычисленный результат своей партии и тоже завершает цикл. В finally обе стороны освобождают сетевые объекты и возвращают доступность кнопок создания и подключения. Затем вызывается показ результата. Порядок сообщений и действий при окончании партии представлен на рисунке 2.4

```
if (isHost)
{
    listener = new TcpListener(IPAddress.Any, port);
    tcpListener = listener;
    listener.Start();
    form.lblConnectionStatus.Text = "Статус: Ожидание игрока";
    form.lblHint.Text = $"Ожидание подключения на порту {port}";

    client = await listener.AcceptTcpClientAsync(cancellation.Token);
    cancellation.Token.ThrowIfCancellationRequested();
    listener.Stop();
    tcpListener = null;
    tcpClient = client;
}
else
{
    client = new TcpClient(serverAddress!.AddressFamily);
    tcpClient = client;
    form.lblConnectionStatus.Text = "Статус: Подключение";
    form.lblHint.Text = $"Подключение к {serverAddress}:{port}";

    using var timeout =
CancellationTokenSource.CreateLinkedTokenSource(cancellation.Token);
    timeout.CancelAfter(TimeSpan.FromSeconds(10));
    await client.ConnectAsync(serverAddress, port, timeout.Token);
}
cancellation.Token.ThrowIfCancellationRequested();
form.lblMyColor.Text = isHost ? "Ваш цвет: Белые" : "Ваш цвет: Чёрные";
form.lblConnectionStatus.Text = "Статус: Подключено";
form.StartGame(isHost);
NetworkStream stream = client.GetStream();
byte[] move = new byte[4];
while (await stream.ReadAsync(move.AsMemory(0, 1), cancellation.Token) != 0)
{
    await stream.ReadExactlyAsync(move.AsMemory(1, 3), cancellation.Token);
    cancellation.Token.ThrowIfCancellationRequested();
    if (move[0] == ResultMessage)
    {
        finalWinner = form.GetWinner();
        break;
    }
    if (!form.ReceiveMove(move))
    {
        throw new IOException("Получен недопустимый ход");
    }
    bool? receivedWinner = form.GetWinner();
    if (receivedWinner != null)
    {
        byte[] result = { ResultMessage, (byte)(receivedWinner.Value ? 1 :
0), 0, 0 };
        await stream.WriteAsync(result.AsMemory(), cancellation.Token);
        cancellation.Token.ThrowIfCancellationRequested();
        finalWinner = receivedWinner;
        break;
    }
}
```

Рисунок 2.4 TCP-обмен при завершении партии

Ручное отключение использует общий источник отмены, закрывает TcpClient и останавливает TcpListener. Отмена позволяет прервать ожидание клиента или данных без необходимости получить от соперника очередной ход. Ссылки на ресурсы обнуляются, а Game переводится в остановленное состояние. Если закрывается сама

форма, сетевые ресурсы освобождаются без попытки обновить уже уничтожаемые элементы окна.

Для каждой операции соединения сохраняется собственный объект отмены. В завершающем блоке проверяется, относится ли текущая общая ссылка именно к этой операции. Это необходимо на границе старой и новой сессии: позднее завершение прежнего асинхронного ожидания не должно очищать ссылки нового подключения. Проверка касается принадлежности сетевых ресурсов конкретному сеансу.

В проекте использовал асинхронные методы с await, чтобы ожидание ввода-вывода не блокировало поток окна. Асинхронность позволяет приостановить метод до завершения операции без синхронного ожидания в обработчике [7]. Она не означает одновременное право двух участников менять доску. Ограничение очередности по-прежнему задаётся Game, а сетевой цикл последовательно применяет полностью прочитанные сообщения.

3. РЕАЛИЗАЦИЯ ПРОГРАММЫ

3.1. Выбор C#, Windows Forms, .NET и Rider

Для реализации выбрал C#, поскольку в одном проекте можно использовать стандартные элементы Windows Forms, объекты предметной области и классы сетевого взаимодействия .NET. Язык позволил выразить состояние шашки небольшим классом, а процедуры соединения оформить как асинхронные методы. При таком подходе не требуется отдельная библиотека для представления четырёх координат или вызова стандартных TCP-операций.

Windows Forms использовал для статического оконного интерфейса. В проекте достаточно формы, панели доски, кнопок клеток, полей адреса и порта, группирующих элементов и текстовых надписей. Возможности сложной графической сцены или произвольного масштабирования для выбранного интерфейса не требуются. Каждая клетка уже обладает событием нажатия, поэтому обработка выбора непосредственно связана с существующим элементом.

Размещение клеток выполнил в дизайнере. Для доски создал 64 кнопки одинакового размера, а затем объединил их единым обработчиком Click.

В файле проекта указал net10.0-windows и использование Windows Forms. Приложение собирается как оконная программа WinExe. Для разработки требуется SDK, а для запуска зависимой от платформы сборки используется настольная среда выполнения .NET. В документации Microsoft эти назначения SDK и Desktop Runtime разделены [8].

JetBrains Rider использовал как среду редактирования C# и работы с проектом. Дизайнер Windows Forms в Rider предназначен для визуального редактирования форм на Windows [9]. Для этой работы он позволил расставить элементы и назначить обработчики событий, сохранив код формы в привычной структуре.

Ресурсы доски и шашек использовал для отображения, а не для определения допустимости хода. Простая белая и чёрная шашки имеют отдельные изображения. Дамку обозначил текстом поверх соответствующего изображения. Поэтому логический переход в дамку не требует загрузки новой внешней графики и выражается изменением IsQueen и текста кнопки.

Для свойств Checker использовал логические значения. IsWhite определяет цвет без строкового сравнения, а IsQueen определяет вид перемещения. Возможность отсутствия объекта в Tag выражается null. В обработчиках применил проверку типа, чтобы отличать занятую клетку от пустой и получать свойства фигуры. Это делает связь между выбранным элементом и объектом явной.

Для последовательного сетевого ввода-вывода использовал стандартные методы NetworkStream. Буфер хода имеет постоянную длину, поэтому сериализация общего объекта с именами полей не понадобилась. Массив непосредственно формируется из координат клеток. На другой стороне эти же байты используются как индексы положения, после чего правила игры решают, допустим ли такой переход.

3.2. Интерфейс и последовательность действий игроков

Главное окно разделил на игровое поле и область управления. Доска размещена слева и занимает квадрат 1000 на 1000 единиц координат интерфейса. Справа находятся параметры подключения и сведения о партии. Над доской показан статус соединения, а в правой части указан цвет пользователя.

Размер клетки составляет 125 на 125. Эти значения использованы не только при размещении элементов, но и в расчёте координат. Поэтому на этапе интерфейса сохранил единые размеры и плотное расположение клеток без дополнительных промежутков между ними.

Основные области окна представлены на рисунке 3.1: доска, параметры соединения, цвет пользователя, ход и счётчики шашек.

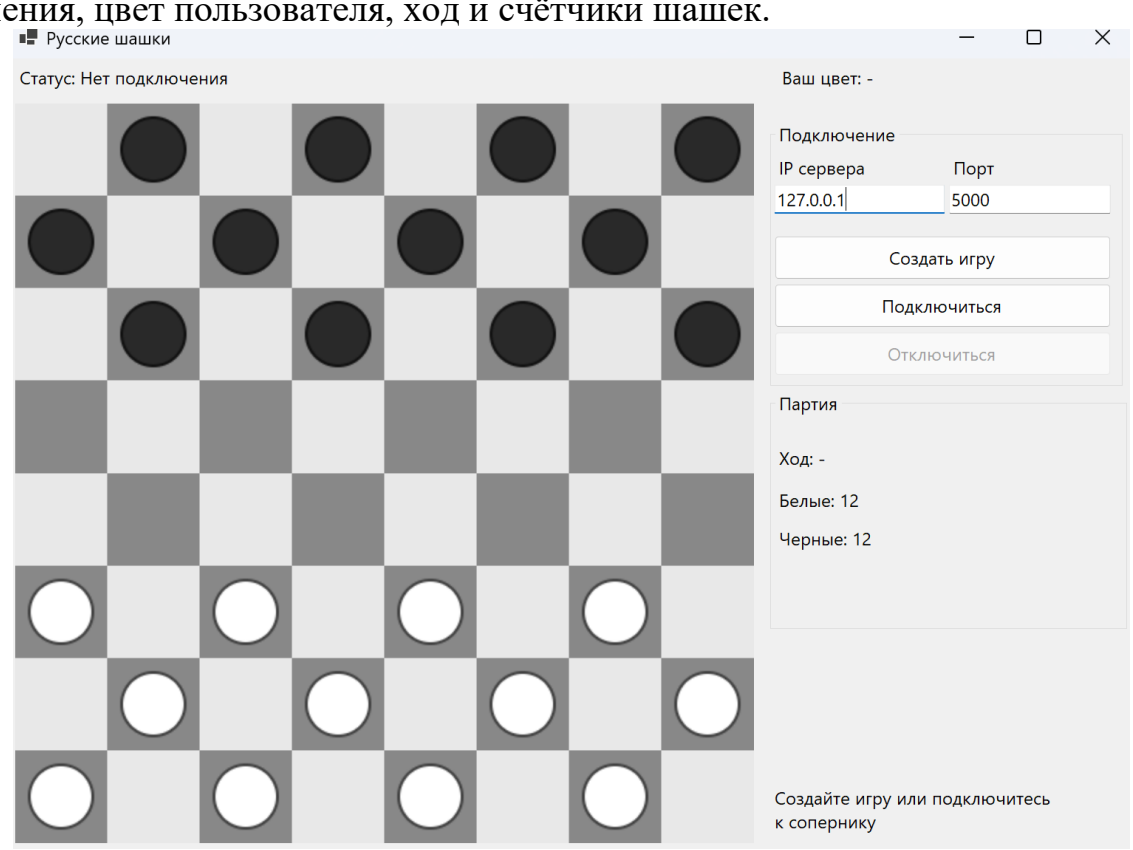


Рисунок 3.1 Главное окно сетевой игры

В группе подключения расположил поле IP-адреса, поле порта и три кнопки: «Создать игру», «Подключиться» и «Отключиться».

Для создания игры пользователь задаёт порт и нажимает соответствующую кнопку. Поле адреса в этом режиме не определяет интерфейс прослушивания, поскольку принимающая сторона использует IPAddress.Any. Программа переходит в ожидание и показывает номер порта в подсказке. Игровые действия ещё недоступны, поскольку второй участник не принят и общая партия не начата.

Подключающийся участник указывает IP-адрес принимающего компьютера и тот же порт. После нажатия «Подключиться» статус показывает выполняемую операцию. Если адрес записан неправильно или порт не является допустимым числом, сообщение появляется до сетевого ожидания. Если параметры приняты, но соединение не установлено за отведённое время, подсказка сообщает об отсутствии ответа.

После успешной связи в обоих окнах формируется начальная позиция. Создателю назначаются белые, подключившемуся пользователю чёрные. Надпись «Ваш цвет» остаётся неизменной в пределах партии, а надпись «Ход» меняется после завершённых действий.

Для перемещения игрок сначала нажимает свою шашку, затем пустую клетку. При нажатии занятого противником поля ход не выполняется: взятие задаётся выбором свободного поля за противником, а не нажатием самой снимаемой фигуры. Если взятие обязательно, подсказка прямо сообщает об этом. Выбор фигуры, у которой нет допустимого взятия, не создаёт перемещения. При наличии нескольких бьющих шашек участник может выбрать одну из них. После начала цепочки подсказка меняется на требование продолжить той же шашкой. Таким образом, текстовое состояние сопровождает фактическое ограничение в алгоритме.

На стороне соперника промежуточное взятие отображается после приёма координат. Пока цепочка не закончена, право ответного хода не появляется. Надпись указывает, что соперник продолжает взятие. Полное сетевое взаимодействие предусмотрел на рисунке 3.2.

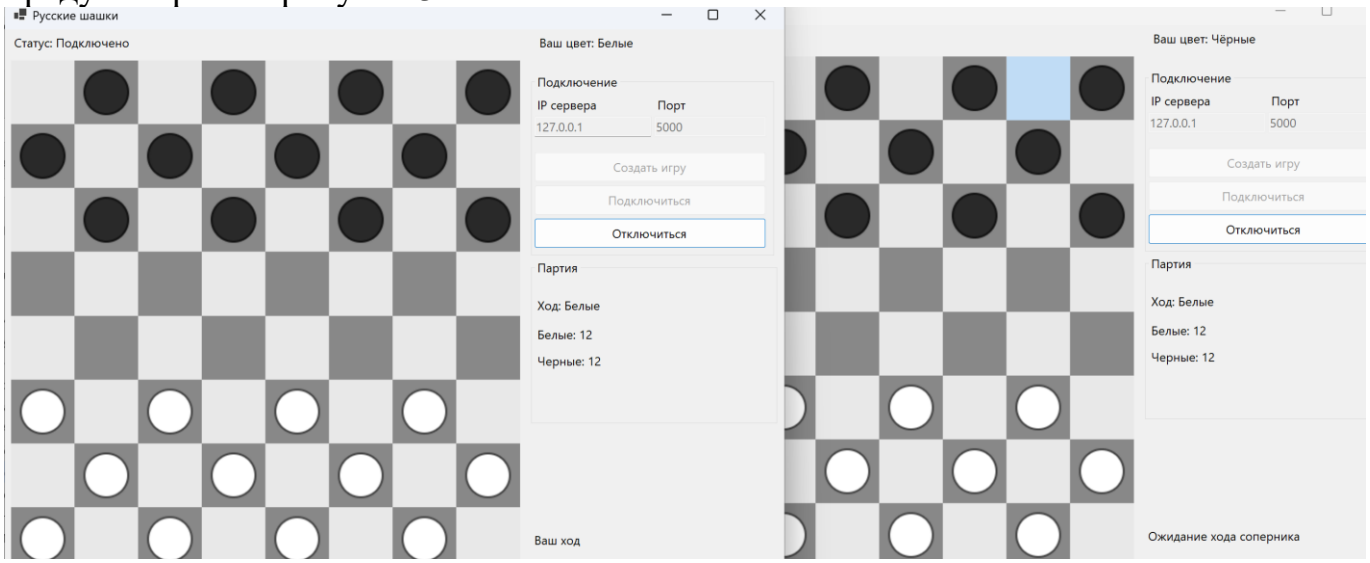


Рисунок 3.2 Отображение сетевой партии у двух участников

В группе партии показываю текущий ход и количество шашек каждого цвета. Счётчики обновляются после завершения обычного хода или всей цепочки взятий. Во время серии взятые фигуры сохраняются до её конца, что соответствует принятому порядку обработки. После их удаления количество пересчитывается по клеткам, содержащим объекты Checker.

Отображение дамки после превращения показано на рисунке 3.3.

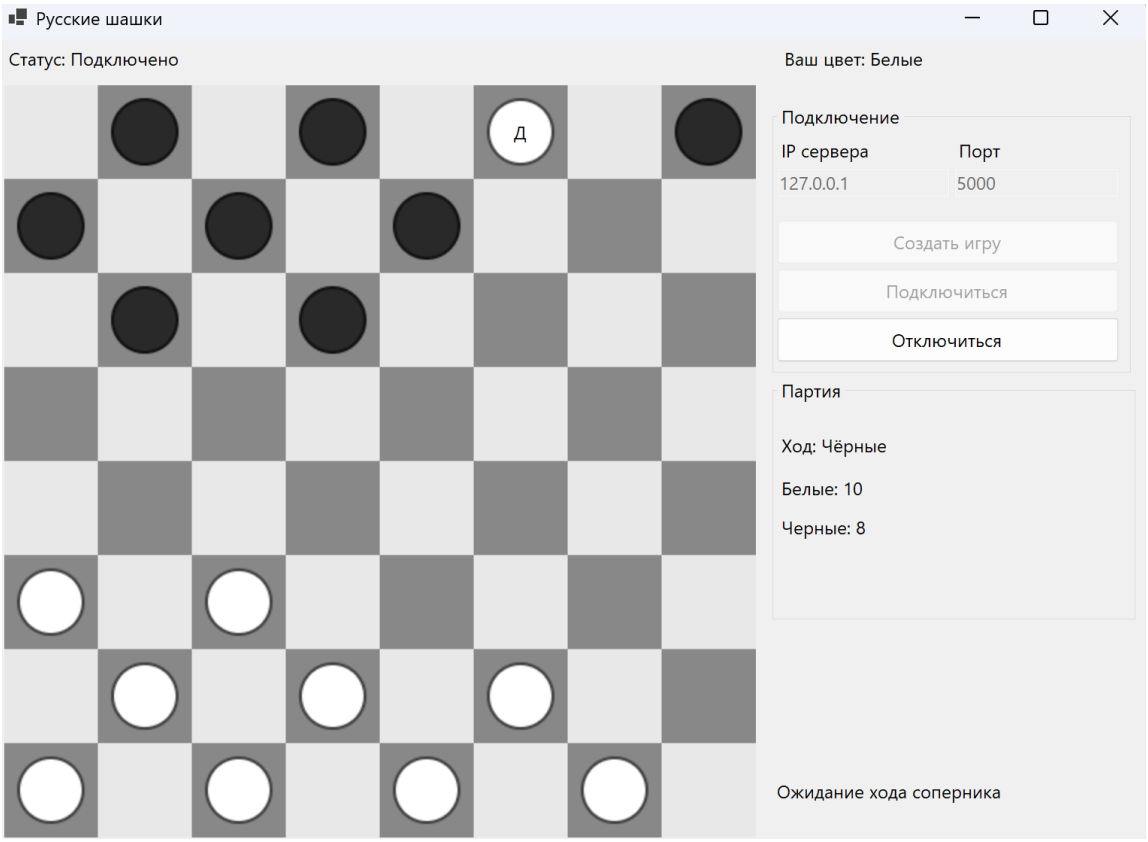


Рисунок 3.3 Отображение дамки на игровом поле

После взятия последней шашки соперника дальнейшие нажатия не должны изменять позицию. Game отмечает конец партии, а Network завершает обмен подтверждением. Затем в каждом окне появляется сообщение с цветом победителя и личным результатом. Окно победившего участника сообщает «Вы победили», окно другого «Вы проиграли».

Сообщения участникам об окончании партии показаны на рисунке 3.4

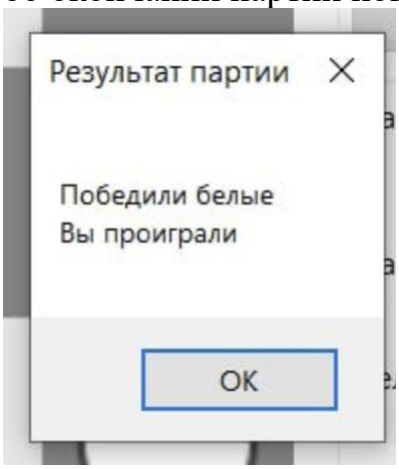


Рисунок 3.4 Показ победителя участникам сетевой игры

Кнопка отключения завершает сессию без продолжения текущей позиции. После её нажатия отменяются ожидающие сетевые операции, закрываются ресурсы, а доска становится недоступной для ходов. Надписи возвращаются к состоянию отсутствия подключения. Удалённая сторона при завершении потока также прекращает игру. При закрытии окна форма вызывает освобождение сетевых ресурсов с отключённым обновлением элементов управления.

Начало следующей партии вызывает повторную инициализацию. Старые выделения, сведения о цепочке, результат и свойства дамок не должны переноситься в новый сеанс. В текущем коде StartGame очищает эти состояния и формирует новые Checker. Отображение прошлой конечной позиции до нового подключения не означает её использование как начальной позиции следующей партии.

ЗАКЛЮЧЕНИЕ

В курсовой работе разработал сетевую компьютерную игру «Русские шашки» для двух пользователей. Создал одно приложение на C# с интерфейсом Windows Forms, в котором доступны режим создания игры и режим подключения. Распределил обязанности между формой, игровой логикой, сетевым классом и объектом шашки.

Реализовал обычные перемещения, обязательные взятия и их последовательное продолжение одной шашкой. Предусмотрел дальние перемещения дамки и превращение простой шашки при достижении последней строки. Превращение внутри взятия учитывается до проверки продолжения, поэтому дальнейшее действие оценивается с актуальными возможностями фигуры. Снятие взятых фигур выполняется после завершения всей цепочки.

Для взаимодействия экземпляров организовал TCP-соединение и обмен четырёхбайтовыми сообщениями. Координаты передаются без графических данных, а получатель применяет общие правила к своей позиции. Обработал частичное чтение потока, чтобы перемещение выполнялось только после получения полного сообщения. Предусмотрел отмену ожидания и освобождение сетевых ресурсов при отключении.

Окончание партии реализовал по отсутствию шашек у соперника. После последнего хода принимающая сторона отправляет подтверждение завершения, затем оба экземпляра закрывают соединение и показывают результат. Победитель определяется локальной игровой логикой, а служебное сообщение обозначает завершение обработки последнего действия. Новое подключение создаёт новую начальную позицию.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Rules of the game : Official Rules of the game in draughts-64 / International Draughts Federation. - URL: https://fmjd64.org/rules-of-the-game/ (дата обращения: 26.09.2026).

2. What is Windows Forms / Microsoft Learn. - URL: https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/ (дата обращения: 26.09.2026).

3. Control.Tag Property / Microsoft Learn. - URL: https://learn.microsoft.com/en-us/dotnet/api/system.windows.forms.control.tag?view=windowsdesktop-10.0 (дата обращения: 26.09.2026).

4. TCP overview / Microsoft Learn. - URL: https://learn.microsoft.com/en-us/dotnet/fundamentals/networking/sockets/tcp-classes (дата обращения: 26.09.2026).

5. NetworkStream.ReadAsync Method / Microsoft Learn. - URL: https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.networkstream.readasync?view=net-10.0 (дата обращения: 26.09.2026).

6. Stream.ReadExactlyAsync Method / Microsoft Learn. - URL: https://learn.microsoft.com/en-us/dotnet/api/system.io.stream.readexactlyasync?view=net-10.0 (дата обращения: 26.09.2026).

7. Asynchronous programming / Microsoft Learn. - URL: https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/ (дата обращения: 29.09.2026).

8. Install .NET on Windows / Microsoft Learn. - URL: https://learn.microsoft.com/en-us/dotnet/core/install/windows (дата обращения: 29.09.2026).

9. Design Windows Forms / JetBrains Rider Documentation. - URL: https://www.jetbrains.com/help/rider/Working_with_Windows_Forms.html (дата обращения: 29.09.2026).